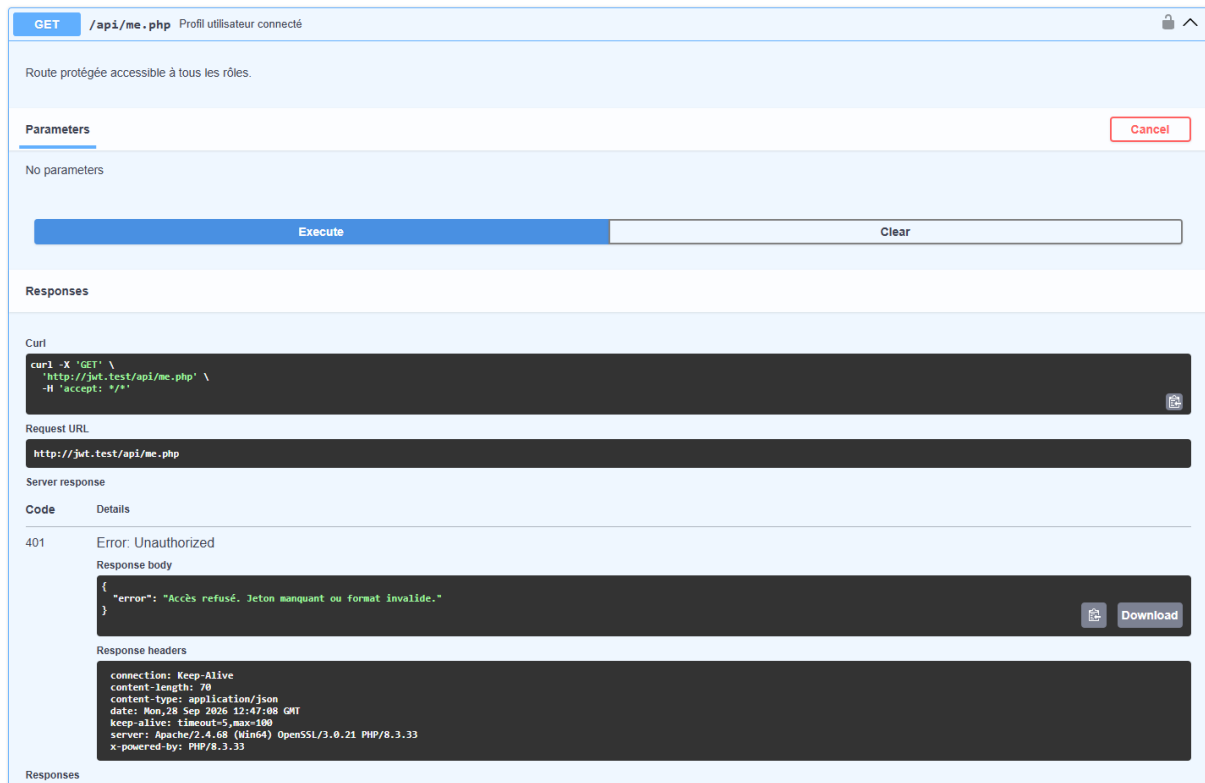


TP JWT

Lien GitHub du TP : <https://github.com/HSL-GliTcheur/Token-JWT>

Pour tester mon API j'ai choisi swagger :

La première requête consiste à demander mon identité sans être connecté



N'ayant pas de token de le header de ma requête, une erreur 401 est renvoyée indiquant l'accès refusé.

Hugo SIMON

En deuxième, j'ai vérifié que la connexion refuse l'accès si les informations ne sont pas bonnes, sans préciser quel champ est mauvais pour éviter d'aider un éventuel attaquant.

Dans cet exemple j'ai retiré le dernier caractère du mot de passe :

POST

/api/login.php

Connexion et obtention du token JWT

^

Route publique. Retourne un jeton JWT valable 15 minutes.

Parameters

Cancel

Reset

No parameters

Request body

required

application/json

▼

Edit Value

Schema

```
{
  "email": "tech1@geotech.fr",
  "password": "secure45"
}
```

Execute

Clear

La réponse :

Code

Details

401

Error: Unauthorized

Response body

```
{
  "error": "Identifiants incorrects."
}
```

Download

Response headers

```
connection: Keep-Alive
content-length: 36
content-type: application/json
date: Mon, 28 Sep 2026 12:49:30 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.68 (Ubuntu) OpenSSL/3.0.21 PHP/8.3.33
x-powered-by: PHP/8.3.33
```

Hugo SIMON

Ensuite j'ai testé la connexion avec tech1@geotech.fr, pour pouvoir obtenir un token temporaire de 15 minutes.

The screenshot shows a REST client interface with the following sections:

- Header:** Method: POST, URL: /api/login.php, Description: Connexion et obtention du token JWT.
- Parameters:** A section with "No parameters" and buttons for "Cancel" and "Reset".
- Request body:** A section with a dropdown menu set to "application/json". Below it, a text area contains the JSON body:

```
{  "email": "tech1@geotech.fr",  "password": "secure456"}
```
- Execute:** A large blue button labeled "Execute".
- Responses:** A table showing the expected response codes and descriptions.

Code	Description	Links
200	Authentification réussie	No links
401	Identifiants invalides	No links

Je choisis la requête en POST sur l'url /api/login.php et je rentre mes informations dans le body de la requête

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
'http://jwt.test/api/login.php' \
-H 'accept: */*' \
-H 'content-type: application/json' \
-d '{
  "email": "tech@geotech.fr",
  "password": "secure456"
}'
```

Request URL

```
http://jwt.test/api/login.php
```

Server response

Code	Details
200	Response body <pre>{ "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJpZCI6ImwiZWZhbmkiOiI0ZmNoMUMnZW90ZmNoLmZyIiwicm9sZSI6IjRlRFqQmOSUNDRU41CjpwYXQ1QjE3OTAtMTg2OTFsImV4cCI6NTc1MDU5OTU5N30.3MgxskqcIGhg3-sDF9rOikK-UfgnnIRA_4AJg_zFDA" }</pre> Download

Response headers

```
connection: Keep-Alive
content-length: 212
content-type: application/json
date: Mon, 20 Sep 2026 12:31:33 GMT
keep-alive: timeout=5,max=100
server: Apache/2.4.68 (Ubuntu) OpenSSL/3.0.21 PHP/8.3.33
x-powered-by: PHP/8.3.33
```

Responses

Code	Description	Links
200	Authentication réussie	No links
401	Identifiants invalides	No links

Ensuite on récupère le token renvoyé par l'API pour l'ajouter dans swagger

Available authorizations

x

bearerAuth (http, Bearer)

Value:

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQifQ.eyJ1bm9udGVudCI6ImFkbWwifQ.eyJpcyI6ImNpdGUiLCJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiIsInR5cCI6IkpXZWQifQ.eyJ1bm9udGVudCI6ImFkbWwifQ.

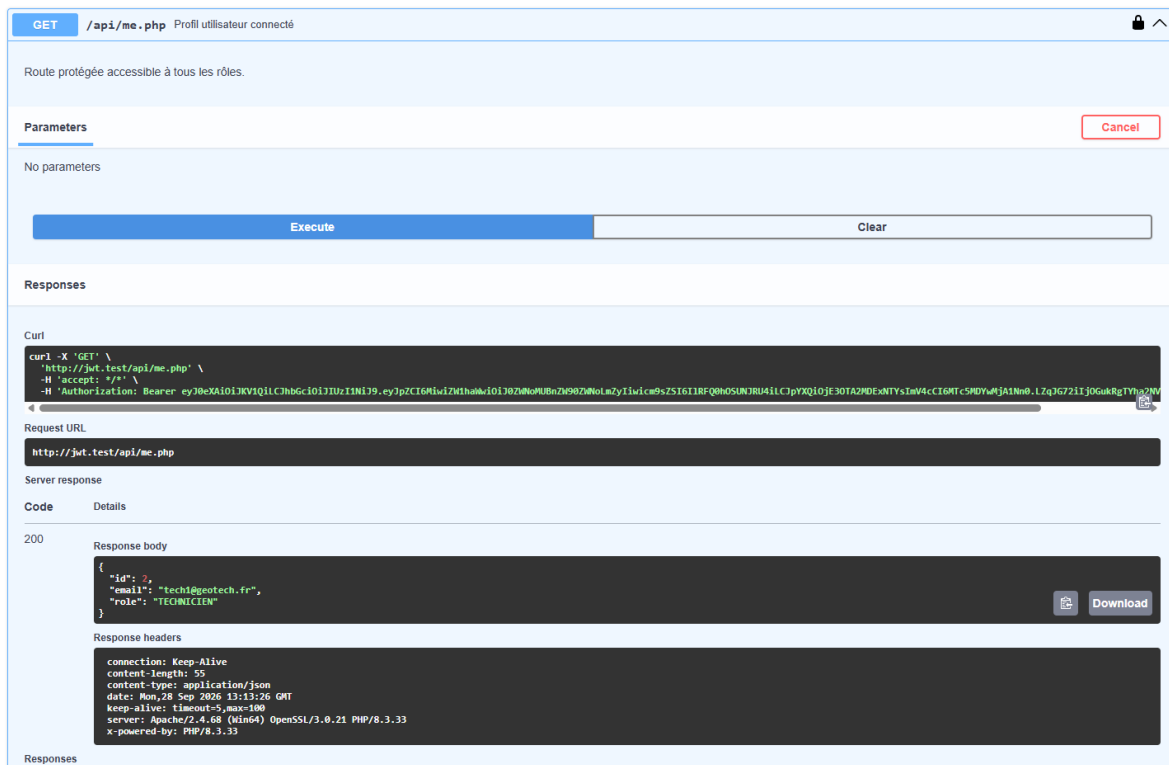
Authorize

Close

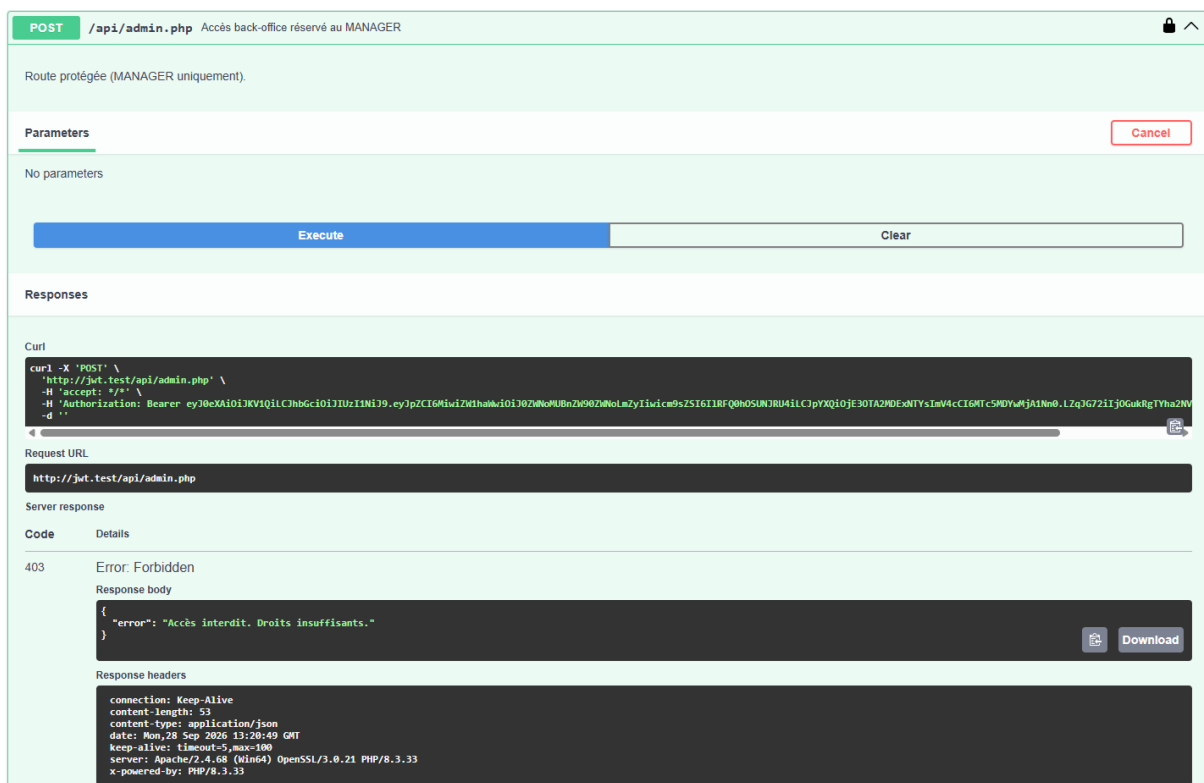
Swagger va conserver le token et le mettre dans le header de chaque requête pour rester identifié

Hugo SIMON

J'ai ensuite fait une requête avec la méthode GET, sur /api/me pour connaître mon identité :



Pour finir, j'ai fait une requête sur `/api/admin` nécessitant le rôle admin, étant technicien j'obtiens une réponse erreur 403 et une erreur dans le corps de la réponse.



Explication du code :

AuthMiddleware.php :

```
1 <?php
2 require_once __DIR__ . '/vendor/autoload.php';
3
4 use Firebase\JWT\JWT;
5 use Firebase\JWT\Key;
6
7 class AuthMiddleware
8 {
9     private static $secretKey = "H15rmyTulVjj2L3xdLTfiKS1H326gqAq1Q8FuADPSyT";
10
11     public static function getSecretKey()
12     {
13         return self::$secretKey;
14     }
15
16     public static function verifyToken()
17     {
18         // getAllheaders() lit l'extérieur de l'enveloppe HTTP
19         // (Les en-têtes reçus par le serveur lors de la requête).
20         $headers = getAllheaders();
21
22         // La syntaxe ?? sert de filet de secours : "cherche dans $headers, sinon dans $_SERVER, sinon mets null".
23         $authHeader = $headers['Authorization'] ?? $_SERVER['HTTP_AUTHORIZATION'] ?? null;
24
25         /* preg_match(...) : analyse le texte de l'en-tête.
26         Bearer\s : cherche le mot "Bearer" suivi d'un espace (\s).
27         (\S+) : capture tout ce qui suit sans espace (c'est notre JWT). */
28         if (!$authHeader || !preg_match('/Bearer\s(\S+)/', $authHeader, $matches)) {
29             http_response_code(401);
30             echo json_encode(["error" => "Accès refusé. Jeton manquant ou format invalide."]);
31             exit();
32         }
33
34         // contient uniquement la clé JWT isolée.
35         $token = $matches[1];
36
37         try {
38             $decoded = JWT::decode($token, new Key(self::$secretKey, 'HS256'));
39             return (array) $decoded;
40         } catch (Exception $e) {
41             http_response_code(401);
42             echo json_encode(["error" => "Jeton invalide ou expiré."]);
43             exit();
44         }
45     }
46
47     public static function requireRole($user, $roleAttendu)
48     {
49         if (!isset($user['role']) || $user['role'] !== $roleAttendu) {
50             http_response_code(403);
51             echo json_encode(["error" => "Accès interdit. Droits insuffisants."]);
52             exit();
53         }
54     }
55 }
```

La fonction `verifyToken` sert à vérifier la signature du token. Du début de la fonction jusqu'au « `try` », on récupère le token. Ensuite, dans le `try`, on effectue la vérification de la signature du token avec `JWT::decode`. Si la signature correspond toujours et qu'aucune modification n'a été effectuée, la variable « `decoded` » prend un tableau associatif avec les valeurs du payload. Il vérifie également si le token a expiré.

Login.php :

```
1  <?php
2  header("Content-Type: application/json");
3  require_once __DIR__ . '/../AuthMiddleware.php';
4  use Firebase\JWT\JWT;
5
6  // Utilisateurs de test
7  $users = [
8      [
9          "id" => 1,
10         "email" => "user1@geotech.fr",
11         "password" => "password123",
12         "role" => "MANAGER"
13     ],
14     [
15         "id" => 2,
16         "email" => "tech1@geotech.fr",
17         "password" => "secure456",
18         "role" => "TECHNICIEN"
19     ]
20 ];
21
22 // Récupération du corps de la requête (JSON)
23 $input = json_decode(file_get_contents('php://input'), true);
24 $email = $input['email'] ?? '';
25 $password = $input['password'] ?? '';
26
27 $authenticatedUser = null;
28 foreach ($users as $u) {
29     if ($u['email'] === $email && $u['password'] === $password) {
30         $authenticatedUser = $u;
31         break;
32     }
33 }
34
35 if (!$authenticatedUser) {
36     http_response_code(401);
37     echo json_encode(["error" => "Identifiants incorrects."]);
38     exit();
39 }
40
41 // Payload JWT : expire dans 15 minutes
42 $payload = [
43     "id" => $authenticatedUser['id'],
44     "email" => $authenticatedUser['email'],
45     "role" => $authenticatedUser['role'],
46     "iat" => time(),
47     "exp" => time() + (15 * 60)
48 ];
49
50 $jwt = JWT::encode($payload, AuthMiddleware::getSecretKey(), 'HS256');
51
52 http_response_code(200);
53 echo json_encode(["token" => $jwt]);
```

On récupère les informations dans le corps de la requête, on teste les informations parmi la table user. Si les identifiants sont incorrects on renvoie une erreur 401 sinon on crée le payload.

Puis à la fin, JWT utilise la clé secrète pour signer le contenu du token et générer un token signé. Ainsi, si une modification est effectuée sur le token, par exemple pour modifier le rôle, la signature ne correspondra plus au contenu du token. Le serveur pourra alors détecter la modification et refuser le token.